

Homework 1 Design Document: In-Memory B+ Tree (C++)
Name: Ben

1. Data Structures

Node representation. I built the whole out of a struct called `Node`, which is used for both leaves and internal nodes. I chose this because it keeps the code short and I never have to cast between two node types. A `Node` has a `bool` called `leaf`, a vector of ints called `keys`, a vector of ints called `ptrs`, a vector of `Node` pointers called `kids`, and a `Node` pointer called `next`. The `ptrs` vector is only used in leaves, where `ptrs[i]` is the pointer that goes with `keys[i]`. The `kids` vector is only used in internal nodes and always has exactly one more entry than `keys`. The `next` pointer links each leaf to the leaf that follows it in key order. The only global state is the root pointer and the integer `D`, which is read from the command line. Keys and pointers are plain ints, and duplicate keys are not allowed.

Capacity. The program is started with `./bplus init d < test.txt`. A node holds at most $2d$ keys, so a leaf holds up to $2d$ key-pointer pairs, and an internal node holds up to $2d$ keys and $2d+1$ children. Every node except the root must keep at least d keys, meaning it is at least half full. The root is allowed to be smaller than that: it can be a leaf with anywhere from 0 to $2d$ keys, or an internal node with at least one key and two children.

Routing rule. In an internal node, the child `kids[i]` holds the keys k that satisfy $\text{keys}[i-1] \leq k < \text{keys}[i]$. This means a key equal to a separator goes to the right of that separator. To pick a child I call `upper_bound` on the node's keys and use the result as the child index, which gives exactly this behavior. All leaves are at the same depth and are chained together through `next`, so a range query only needs to find the first leaf and then walk along the chain.

Page management. In the assignment everything stays in memory, so there are no pages, no index file, and no disk I/O counters. A "page" in my program is a heap-allocated `Node`. I create one with `new` whenever a node splits, and then I free it with `delete` when two nodes merge or when the root collapses into its only child. Just before a split, a node temporarily holds $2d+1$ keys. Because the keys live in a vector, this overflow is easy to allow: I insert first, check the size, and split afterward.

2. Overall Design and Algorithms

All of the code is in one file and is split into small functions. The function `childIndex` and the function `findLeaf` handle navigation. Then there is `search`, `insert` (which uses a recursive helper called `ins`), `remove` (which uses `del`, `fixChild`, and `mergeNodes`), `rangeSearch`, `printTree` and `printStatistics` (which both use one shared function called `traverse`), and finally `main`. The `main` function creates an empty root leaf, reads commands from standard input one line at a time, and calls the matching function. Command names are not case sensitive, and a line that does not parse is skipped. Each operation prints in the message format given in the assignment. Search, insert, and delete each visit $O(\log n)$ nodes, because the tree height is logarithmic and each step only touches one node plus at most one sibling.

Search. I start at the root and follow the child chosen by the routing rule until I reach a leaf. Then I binary-search the keys in that leaf and print either the found message with the pointer or the not-found message.

Insert. Insertion is recursive. When the recursion reaches a leaf, the pair is placed at its sorted position. If the key is already there, the program prints the duplicate message and changes nothing. If the leaf now has $2d+1$ keys, it splits: the left node keeps $d+1$ keys, the new right node gets d keys, the leaf chain is relinked so the new node sits right after the old one, and the first key of the right node is passed back up to the parent. The parent inserts that key and the new child pointer. If the parent overflows too, it splits in the same way, except that its middle key moves up and is not kept in either half, so each half ends up with d keys. If the split reaches the root, I create a new root with a single key and two children, and the tree grows by one level.

Delete. Deletion is also recursive. The pair is removed from its leaf, and on the way back up each parent checks the child it just descended into. If that child now has fewer than d keys, the parent tries to fix it. In my code a child that has a left neighbor looks only at that left sibling, and the leftmost child looks only at its right sibling. If that sibling has more than d keys, the child borrows one entry from it and the separator key in the parent is updated. In a leaf this means moving the key and pointer across and copying the new first key up. In an internal node it is a rotation: the separator comes down into the child and the sibling's edge key goes up. If the sibling has no spare entries, the two nodes merge into one, the extra node is freed, and the separator is removed from the parent, which may then underflow in turn. For internal nodes the separator is pulled down into the merged node, and for leaves it is simply dropped and

the next pointer is repaired. Because I only check one sibling, there are cases where a merge happens even though the other sibling could have lent a key. The tree is still valid in those cases, since every node still has at least d keys. Finally, if the root is left with no keys, its only child becomes the new root and the tree gets one level shorter.

Range search. I find the leaf that would hold k_1 and scan forward along the leaf chain, collecting every pair whose key is at least k_1 , and stop as soon as I see a key larger than k_2 . If any pairs were collected, I print the word found followed by the pairs, one per line. Otherwise I print the no-records message. A reversed range, where k_1 is larger than k_2 , naturally returns nothing.

Print and statistics. PRINT does a level-by-level traversal starting from the root. It prints one line per level and one pair of square brackets per node, with (key: pointer) entries inside leaves and (key) entries inside internal nodes. PRINT STATISTICS uses the same traversal without printing, and reports the tree height as the number of levels, the total number of nodes, and the total number of keys stored in the leaves. Counting only leaf keys matters because internal keys are copies used for routing and are not extra records.

3. Testing

The submitted test.txt is the sample commands from the assignment. With $d = 2$ every key fits in a single node, so the tree stays one level tall. The three inserts, the search, the range search from 5 to 20, and the delete of key 10 all print the expected messages, and the final PRINT shows Level 0: [(5: 56743) (20: 37658)]. With $d = 1$ the same file forces a split, so the tree reaches height 2 with three nodes, which is a quick way to see that the split and the new root are working. Beyond the sample, I ran the some more tests.

- The assignment sample with $d = 1$ and with $d = 2$, comparing every output line against the format in the assignment.
- A larger scripted test of about 90 commands with $d = 2$. It had 20 ascending inserts that build a three-level tree, extra inserts that split a middle leaf, duplicate inserts, found and not-found searches, and several range searches: a normal range, the full range, an empty range, a range at the end of the tree, and a reversed range.
- Hand tracing of splits, borrows, and merges, which I worked out on paper and compared with the PRINT output after each step.

Together these cover every operation and every structural case in the code: leaf split, internal split, root split, borrowing from each side, leaf merge, internal merge, and root collapse.